Assignment 1

Design a program to maintain a student database that performs the following tasks:

1. Add and store student details (ID, Name, CGPA) using dynamically allocated memory.

2. Expand the student list using realloc() as new entries are added.

3. Implement Linear Search and Binary Search to find student records by ID.

4. Implement at least two Sorting Algorithms (Bubble Sort, Selection Sort, or Insertion Sort) to sort

student records by:

o Name (Alphabetically)

o CGPA (Ascending/Descending)

5. Analyze and compare the performance of search operations before and after sorting.

```
#include <iostream>
#include <cstring>
#include <cstdlib>
#include <algorithm>
using namespace std;

struct Student {
   int id;
   char name[50];
```

```
    float cgpa;
};

Student* students = nullptr;
int studentCount = 0;

// Function to add student using realloc
void addStudent() {
    students = (Student*)realloc(students, (studentCount + 1) * sizeof(Student));
    if (!students) {
        cout << "Memory allocation failed!" << endl;
        return;
    }

    cout << "\nEnter ID: ";
    cin >> students[studentCount].id;
    cout << "Enter Name: ";
    cin.ignore();
    cin.getline(students[studentCount].name, 50);
    cout << "Enter CGPA: ";
    cin >> students[studentCount].cgpa;

    studentCount++;
```

```
    cout << "Student added successfully!\n";
}

// Linear Search by ID
int linearSearch(int id) {
    for (int i = 0; i < studentCount; ++i) {
        if (students[i].id == id)
            return i;
    }
    return -1;
}

// Binary Search by ID (after sorting by ID)
int binarySearch(int id) {
    int left = 0, right = studentCount - 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (students[mid].id == id)
            return mid;
        else if (students[mid].id < id)
            left = mid + 1;
```

```
        else
            right = mid - 1;
    }
    return -1;
}

// Bubble Sort by CGPA
void bubbleSortByCGPA(bool ascending = true) {
    for (int i = 0; i < studentCount - 1; ++i) {
        for (int j = 0; j < studentCount - i - 1; ++j) {
            if ((ascending && students[j].cgpa > students[j + 1].cgpa) ||
                (!ascending && students[j].cgpa < students[j + 1].cgpa)) {
                swap(students[j], students[j + 1]);
            }
        }
    }
}

// Selection Sort by Name
void selectionSortByName() {
    for (int i = 0; i < studentCount - 1; ++i) {
        int minIdx = i;
        for (int j = i + 1; j < studentCount; ++j) {
```

```
            if (strcmp(students[j].name, students[minIdx].name) < 0) {
                minIdx = j;
            }
        }
        if (minIdx != i) {
            swap(students[i], students[minIdx]);
        }
    }
}

// Display all student records
void displayAll() {
    cout << "\n--- Student Records ---\n";
    for (int i = 0; i < studentCount; ++i) {
        cout << "ID: " << students[i].id << ", Name: " << students[i].name
             << ", CGPA: " << students[i].cgpa << endl;
    }
}

// Analyze and compare Linear vs Binary Search
void searchAnalysis(int id) {
    cout << "\n[Linear Search - Unsorted] ";
    int lin = linearSearch(id);
```

```
    if (lin != -1)
        cout << "Found at index " << lin << ": " << students[lin].name << endl;
    else
        cout << "Not Found\n";

    // Sort by ID for Binary Search
    sort(students, students + studentCount, [](Student a, Student b) {
        return a.id < b.id;
    });

    cout << "[Binary Search - After Sorting by ID] ";
    int bin = binarySearch(id);
    if (bin != -1)
        cout << "Found at index " << bin << ": " << students[bin].name << endl;
    else
        cout << "Not Found\n";
}

// Main menu
int main() {
    int choice, searchId;
    while (true) {
        cout << "\nMenu:\n";
```

```
cout << "1. Add Student\n";
cout << "2. Display All Students\n";
cout << "3. Linear & Binary Search by ID\n";
cout << "4. Sort by CGPA (Bubble Sort)\n";
cout << "5. Sort by Name (Selection Sort)\n";
cout << "6. Exit\n";
cout << "Enter choice: ";
cin >> choice;

switch (choice) {
    case 1:
        addStudent();
        break;
    case 2:
        displayAll();
        break;
    case 3:
        cout << "Enter ID to search: ";
        cin >> searchId;
        searchAnalysis(searchId);
        break;
    case 4:
        bubbleSortByCGPA();
```

```
            cout << "Sorted by CGPA (Ascending).\n";
            break;
        case 5:
            selectionSortByName();
            cout << "Sorted by Name (Alphabetically).\n";
            break;
        case 6:
            free(students);
            cout << "Exiting...\n";
            return 0;
        default:
            cout << "Invalid choice.\n";
      }
   }
}
```